

Práctica N° 12: Generación de Curvas Paramétricas en 3D Evaluación de Curvas de Bézier Cúbicas en el Vertex Shader

1. Objetivos

- Comprender el fundamento matemático de las curvas paramétricas en el espacio tridimensional, centrándose en las curvas de Bézier cúbicas.
- Aprender a paralelizar la evaluación de ecuaciones polinomiales delegando el cálculo matemático al Vertex Shader en lugar de la CPU.
- Utilizar variables de tipo **uniform** para transferir puntos de control dinámicos desde la aplicación en C++ hacia el pipeline programable de la GPU.

2. Conceptos Teóricos

2.1. Curvas Paramétricas en 3D

A diferencia de las funciones explícitas tradicionales $y = f(x)$, una curva libre en el espacio tridimensional se representa convenientemente mediante una formulación paramétrica. En este enfoque, cada coordenada espacial (x, y, z) se expresa de manera independiente en función de un único parámetro independiente t :

$$P(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}, \quad \text{donde } t \in [t_{min}, t_{max}] \text{ (usualmente } [0, 1])$$

2.2. Curvas de Bézier Cúbicas

Una curva de Bézier cúbica está definida rigurosamente por cuatro puntos de control en el espacio: P_0, P_1, P_2 y P_3 . Los puntos inicial (P_0) y final (P_3) determinan los extremos de la curva, mientras que los puntos intermedios (P_1, P_2) actúan como imanes que definen los vectores tangentes de entrada y salida, modelando la curvatura.

La ecuación matemática que define un punto sobre la curva para un valor dado de t es:

$$P(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3, \quad t \in [0, 1]$$

Esta formulación se fundamenta en los **Polinomios de Bernstein** de grado 3, denotados como $B_{i,3}(t)$:

$$B_{0,3}(t) = (1-t)^3, \quad B_{1,3}(t) = 3(1-t)^2 t, \quad B_{2,3}(t) = 3(1-t) t^2, \quad B_{3,3}(t) = t^3$$

De tal forma que la curva se puede compactar como la combinación lineal:

$$P(t) = \sum_{i=0}^3 B_{i,3}(t) P_i$$

2.3. Evaluación en GPU (Vertex Shader) vs CPU

El enfoque tradicional consiste en calcular el conjunto de puntos de la curva en la CPU mediante un bucle secuencial, almacenar los vértices resultantes en un arreglo y enviarlos a la GPU. Aunque funcional, este esquema desperdicia el paralelismo masivo de la GPU.

En el pipeline gráfico moderno, podemos enviar únicamente el parámetro flotante t como un atributo de vértice elemental (por ejemplo, 100 valores distribuidos uniformemente entre 0,0 y 1,0) y pasar los 4 puntos de control como variables `uniform`. El Vertex Shader procesará de forma simultánea e independiente cada valor de t , calculando instantáneamente la posición geométrica exacta en el espacio.

3. Desarrollo de la Práctica

A continuación se detalla la implementación utilizando C++ junto con las bibliotecas GLEW, GLFW y GLM.

3.1. Código del Vertex Shader (curve.vert)

Este shader recibe el atributo escalar `aT` y calcula la interpolación polinomial de Bernstein en paralelo utilizando los puntos de control provistos por la CPU.

```
1 #version 330 core
2 layout (location = 0) in float aT; // Parametro t asignado a este
   vertice
3
4 // Puntos de control recibidos desde C++
5 uniform vec3 P0;
6 uniform vec3 P1;
7 uniform vec3 P2;
8 uniform vec3 P3;
9
10 // Matrices de transformacion tipicas
11 uniform mat4 model;
12 uniform mat4 view;
13 uniform mat4 projection;
14
15 out float tVector; // Transferencia de parametro al Fragment Shader
16
17 void main() {
18     float t = aT;
19     float u = 1.0 - t;
20
21     // Calculo de los Polinomios de Bernstein
22     float b0 = u * u * u;
23     float b1 = 3.0 * u * u * t;
24     float b2 = 3.0 * u * t * t;
25     float b3 = t * t * t;
26
27     // Posicion final calculada en el espacio de modelado
28     vec3 evaluatedPos = b0 * P0 + b1 * P1 + b2 * P2 + b3 * P3;
29
30     gl_Position = projection * view * model * vec4(evaluatedPos, 1.0);
31     tVector = t; // Enviamos t para generar efectos de color variables
32 }
```

Listing 1: Vertex Shader para Evaluación Paramétrica en GLSL 3.3

3.2. Código del Fragment Shader (curve.frag)

El fragment shader recibe de forma interpolada el valor de t para generar una transición cromática procedural a lo largo de la curva.

```
1 #version 330 core
2 in float tVector;
3 out vec4 FragColor;
4
5 void main() {
6     // Genera un degradado desde cian (t=0) hacia magenta (t=1)
7     vec3 color = mix(vec3(0.0, 1.0, 1.0), vec3(1.0, 0.0, 1.0), tVector);
8     FragColor = vec4(color, 1.0);
9 }
```

Listing 2: Fragment Shader con Degradado Temporal en GLSL 3.3

3.3. Código de la Aplicación Principal (main_curves.cpp)

```
1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/matrix_transform.hpp>
5 #include <glm/gtc/type_ptr.hpp>
6 #include <iostream>
7 #include <vector>
8
9 const GLuint WIDTH = 800, HEIGHT = 600;
10
11 // Shaders integrados como texto para ejecucion directa
12 const char* vertexShaderSource = "#version 330 core\n"
13 "layout (location = 0) in float aT;\n"
14 "uniform vec3 P0; uniform vec3 P1; uniform vec3 P2; uniform vec3 P3;\n"
15 "uniform mat4 model; uniform mat4 view; uniform mat4 projection;\n"
16 "out float tVector;\n"
17 "void main() {\n"
18 "    float t = aT; float u = 1.0 - t;\n"
19 "    vec3 pos = (u*u*u)*P0 + (3.0*u*u*t)*P1 + (3.0*u*t*t)*P2 + (t*t*t)*\n"
20 "    P3;\n"
21 "    gl_Position = projection * view * model * vec4(pos, 1.0);\n"
22 "    tVector = t;\n"
23 "}\n\n0";
24
25 const char* fragmentShaderSource = "#version 333 core\n"
26 "in float tVector;\n"
27 "out vec4 FragColor;\n"
28 "void main() {\n"
29 "    vec3 color = mix(vec3(0.0, 1.0, 1.0), vec3(1.0, 0.0, 1.0), tVector)\n"
30 "    ;\n"
31 "    FragColor = vec4(color, 1.0);\n"
32 "}\n\n0";
33
34 GLuint compileShaders() {
35     GLuint vs = glCreateShader(GL_VERTEX_SHADER);
36     glShaderSource(vs, 1, &vertexShaderSource, NULL);
37     glCompileShader(vs);
38
39     GLuint fs = glCreateShader(GL_FRAGMENT_SHADER);
```

```

38     glShaderSource(fs, 1, &fragmentShaderSource, NULL);
39     glCompileShader(fs);
40
41     GLuint program = glCreateProgram();
42     glAttachShader(program, vs);
43     glAttachShader(program, fs);
44     glLinkProgram(program);
45     return program;
46 }
47
48 int main() {
49     if (!glfwInit()) return -1;
50     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
51     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
52     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
53
54     GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Practica 12:
Curvas Parametricas 3D", NULL, NULL);
55     if (!window) { glfwTerminate(); return -1; }
56     glfwMakeContextCurrent(window);
57
58     glewExperimental = GL_TRUE;
59     if (glewInit() != GLEW_OK) return -1;
60
61     glEnable(GL_DEPTH_TEST);
62     GLuint shaderProgram = compileShaders();
63
64     // Generar datos del parametro t de 0 a 1 con alta densidad (200
puntos)
65     std::vector<float> tSegments;
66     int numSegments = 200;
67     for (int i = 0; i <= numSegments; ++i) {
68         tSegments.push_back((float)i / (float)numSegments);
69     }
70
71     GLuint VAO, VBO;
72     glGenVertexArrays(1, &VAO);
73     glGenBuffers(1, &VBO);
74
75     glBindVertexArray(VAO);
76     glBindBuffer(GL_ARRAY_BUFFER, VBO);
77     glBufferData(GL_ARRAY_BUFFER, tSegments.size() * sizeof(float),
tSegments.data(), GL_STATIC_DRAW);
78
79     // El atributo es simplemente un float unificado (parametro t)
80     glVertexAttribPointer(0, 1, GL_FLOAT, GL_FALSE, sizeof(float), (void
*)0);
81     glEnableVertexAttribArray(0);
82
83     // Definicion estatica inicial de los Puntos de Control en 3D
84     glm::vec3 p0(-10.0f, -5.0f, 0.0f);
85     glm::vec3 p1(-5.0f, 10.0f, 5.0f);
86     glm::vec3 p2( 5.0f, -10.0f, -5.0f);
87     glm::vec3 p3( 10.0f, 5.0f, 0.0f);
88
89     while (!glfwWindowShouldClose(window)) {
90         glClearColor(0.08f, 0.08f, 0.12f, 1.0f);
91         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

```

```

92     glUseProgram(shaderProgram);
93
94     // Matrices de Visualizacion
95     glm::mat4 model = glm::mat4(1.0f);
96     // Peque a rotacion global automatica para apreciar la
97     naturaleza 3D de la curva
98     model = glm::rotate(model, (float)glfwGetTime() * 0.3f, glm::
99     vec3(0.0f, 1.0f, 0.0f));
100
101     glm::mat4 view = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f,
102     0.0f, -25.0f));
103     glm::mat4 projection = glm::perspective(glm::radians(45.0f), (
104     float)WIDTH / (float)HEIGHT, 0.1f, 100.0f);
105
106     // Vincular matrices Uniforms
107     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "model"),
108     1, GL_FALSE, glm::value_ptr(model));
109     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"),
110     1, GL_FALSE, glm::value_ptr(view));
111     glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "
112     projection"), 1, GL_FALSE, glm::value_ptr(projection));
113
114     // Vincular puntos de control Uniforms
115     glUniform3fv(glGetUniformLocation(shaderProgram, "P0"), 1, glm::
116     value_ptr(p0));
117     glUniform3fv(glGetUniformLocation(shaderProgram, "P1"), 1, glm::
118     value_ptr(p1));
119     glUniform3fv(glGetUniformLocation(shaderProgram, "P2"), 1, glm::
120     value_ptr(p2));
121     glUniform3fv(glGetUniformLocation(shaderProgram, "P3"), 1, glm::
122     value_ptr(p3));
123
124     // Dibujar la curva parametrica mapeada como una tira continua
125     de lineas
126     glBindVertexArray(VAO);
127     glDrawArrays(GL_LINE_STRIP, 0, tSegments.size());
128
129     glfwSwapBuffers(window);
130     glfwPollEvents();
131 }
132
133 glDeleteVertexArrays(1, &VAO);
134 glDeleteBuffers(1, &VBO);
135 glDeleteProgram(shaderProgram);
136 glfwTerminate();
137 return 0;
138 }

```

Listing 3: Programa Principal en C++ para Renderizado de Curvas

4. Actividades / Ejercicio Propuesto

A partir del código base proporcionado, el estudiante deberá desarrollar la siguiente extensión avanzada:

1. **Curva de Bézier Compuesta de Dos Segmentos (C^0 y C^1):** Construya una curva

compuesta integrada por dos tramos de Bézier cúbicos correlativos. El primer tramo estará definido por los puntos A_0, A_1, A_2, A_3 y el segundo tramo por B_0, B_1, B_2, B_3 .

2. **Restricción de Continuidad:** Para garantizar una transición geométrica perfectamente suave (C^1), la aplicación en C++ debe calcular automáticamente la posición del punto de control B_1 basándose en la posición de los últimos nodos del primer tramo, cumpliendo estrictamente con la relación vectorial:

$$B_0 = A_3 \quad \text{y} \quad B_1 = A_3 + (A_3 - A_2)$$

3. **Animación Interactiva de Nodos en 3D:** Permita al usuario modificar dinámicamente la posición del punto de control extremo A_0 o B_3 en el espacio tridimensional utilizando las teclas UP, DOWN, LEFT, RIGHT combinadas con modificadores de profundidad (teclas W y S), visualizando en tiempo real la deformación síncrona de toda la curva compuesta sin quiebres visuales.

5. Rúbrica de Evaluación

El profesor de laboratorio asignará las calificaciones parciales de acuerdo a la siguiente matriz estructurada sobre un puntaje total de 20 puntos:

Criterio	Descripción	Puntaje Máx.
Matemática del Pipeline	Sustentación formal en el reporte sobre la duplicación del parámetro t y la correcta transferencia modular a través del Vertex Shader.	5 pts
Continuidad C^1 Estricta	Implementación matemática correcta del cálculo de derivación para acoplar suavemente ambos segmentos sin discontinuidad.	5 pts
Control Dinámico 3D	Configuración exitosa de la interactividad del teclado para alterar las coordenadas x, y, z de un nodo y actualizar los uniforms .	4 pts
Reporte e Informe Técnico	Calidad del documento presentado: contiene explicaciones claras, diagramas algorítmicos propios y capturas etiquetadas.	4 pts
Buenas Prácticas C++	Gestión limpia del ciclo de vida de los Shaders, inicialización semántica rigurosa del buffer de atributos de t y modularidad.	2 pts

Cuadro 1: Rúbrica oficial para la evaluación de la práctica de curvas paramétricas.